alpine_summit_lock_pricer.R

adria

2026-03-09

```r
################################################################################
#  ALPINE SUMMIT-LOCK ASIAN KNOCK-OUT CALL
#  Monte Carlo Pricing under the Risk-Neutral Measure Q
#
#  Product combines three exotic features:
#    1. Summit Lock-In (Memory Ratchet)  -- locks a fraction of intrinsic value
#    2. Asian Tail Settlement            -- terminal payoff uses 3-month average
#    3. Discrete Down-and-Out Barrier    -- knockout if stock breaches floor
#
#  Underlying dynamics (GBM under Q):
#    dS_t = r S_t dt + sigma S_t dW_t^Q
#
#  Author : Adriano Lama
#  Date   : March 2026
################################################################################
```

```r
# 0.  REPRODUCIBILITY
```

```r
set.seed(42)
```

```r
# 1.  MARKET & PRODUCT PARAMETERS
```

```r
S0    <- 100        # Initial spot price (CHF or USD)
K     <- 100        # At-the-money strike
B     <- 70         # Down-and-out barrier (30% below spot)
beta  <- 0.50       # Lock-in fraction (50% of intrinsic is guaranteed)
r     <- 0.02       # Continuous risk-free rate (2%)
sigma <- 0.20       # Annualised volatility (20%)
T_mat <- 2          # Maturity in years
N_sim <- 100000     # Number of Monte Carlo paths
M     <- 24         # Number of discrete monitoring dates (monthly)

# Derived quantities
dt    <- T_mat / M                      # Time step between monitors
t_vec <- seq(dt, T_mat, by = dt)        # Monitoring times: t_1, ..., t_24

cat("==============================================================\n")
```

```
## ==============================================================
```

```r
cat("  ALPINE SUMMIT-LOCK ASIAN KNOCK-OUT CALL -- MC Pricer\n")
```

```
##   ALPINE SUMMIT-LOCK ASIAN KNOCK-OUT CALL -- MC Pricer
```

```r
cat("================================================================\n\n")
```

```
## ================================================================
```

```r
cat(sprintf("  S0 = %.0f | K = %.0f | B = %.0f | beta = %.2f\n", S0, K, B, beta))
```

```
##   S0 = 100 | K = 100 | B = 70 | beta = 0.50
```

```r
cat(sprintf("  r  = %.2f | sigma = %.2f | T = %.1f yr | M = %d dates\n", r, sigma, T_mat, M))
```

```
##   r  = 0.02 | sigma = 0.20 | T = 2.0 yr | M = 24 dates
```

```r
cat(sprintf("  N  = %s paths\n\n", format(N_sim, big.mark = ",", scientific = FALSE)))
```

```
##   N  = 100,000 paths
```

```r
# 2.  SIMULATE GBM PATHS -- EXACT CLOSED-FORM SOLUTION
```

```r
# Under Q, the exact solution at each monitoring date is:
#
#   S(t_i) = S(t_{i-1}) * exp((r - 0.5*sigma^2)*dt + sigma*sqrt(dt)*Z_i)
#
# where Z_i ~ N(0,1) i.i.d.  This is NOT Euler discretisation; it is the
# exact log-normal transition of GBM applied at each discrete step.
```

```r
# Draw all random innovations at once: N_sim rows x M columns
Z <- matrix(rnorm(N_sim * M), nrow = N_sim, ncol = M)

# Compute log-returns per step
drift_per_step <- (r - 0.5 * sigma^2) * dt
vol_per_step   <- sigma * sqrt(dt)
log_returns    <- drift_per_step + vol_per_step * Z   # N_sim x M

# Cumulate log-returns and exponentiate to get price levels
#   Column j of S corresponds to S(t_j) for j = 1, ..., M
log_S <- t(apply(log_returns, 1, cumsum))             # cumulative log-returns
S     <- S0 * exp(log_S)                              # N_sim x M price matrix

cat("  [OK] Simulated", format(N_sim, big.mark = ",", scientific = FALSE), "paths x",
    M, "monitoring dates\n\n")
```

```
##   [OK] Simulated 100,000 paths x 24 monitoring dates
```

```r
# 3.  COMPUTE PATH-DEPENDENT PAYOFF COMPONENTS
```

```r
# -- 3a. Down-and-Out Barrier (discrete monitoring at all 24 dates) ----
#    The option is knocked out (payoff = 0) if the stock price falls to or
#    below the barrier B at ANY monitoring date.
path_min         <- apply(S, 1, min)        # min price on each path
barrier_survived <- (path_min > B)           # TRUE if never breached

n_knocked <- sum(!barrier_survived)
cat(sprintf("  Barrier breaches: %s / %s paths (%.1f%%)\n",
            format(n_knocked, big.mark = ","),
            format(N_sim, big.mark = ",", scientific = FALSE),
            100 * n_knocked / N_sim))
```

```
##   Barrier breaches: 16,927 / 100,000 paths (16.9%)
```

```r
# -- 3b. Summit Lock-In Floor (months 1-21 = accumulation phase) ------
#    At each accumulation date t_i (i = 1,...,21), if S(t_i) > K, the holder
#    locks in a guaranteed floor equal to beta * (S(t_i) - K).
#    The BEST (highest) lock-in across all accumulation dates is remembered.
S_accum      <- S[, 1:21]                                  # Accumulation prices
lock_in_vals <- beta * pmax(S_accum - K, 0)                # Element-wise floors
floor_locked <- apply(lock_in_vals, 1, max)                # Best lock-in per path

cat(sprintf("  Paths with positive lock-in: %s (%.1f%%)\n",
            format(sum(floor_locked > 0), big.mark = ","),
            100 * mean(floor_locked > 0)))
```

```
##   Paths with positive lock-in: 87,811 (87.8%)
```

```r
# -- 3c. Asian Tail Average (months 22, 23, 24 = settlement phase) ----
#    The terminal reference price is the arithmetic average of the stock
#    price at the last three monitoring dates.
S_tail       <- S[, 22:24]
A            <- rowMeans(S_tail)            # Asian tail average per path

# -- 3d. Asian Call Component ----------------------------------------
asian_payoff <- pmax(A - K, 0)

# -- 3e. Final Maturity Payoff ---------------------------------------
#    P_T = 1_{survived} * max( asian_payoff, floor_locked )
#
#    The holder receives the BETTER of:
#      (i)  the Asian tail call payoff, or
#      (ii) the highest locked-in floor from the accumulation phase,
#    provided the barrier was never breached.
payoff <- barrier_survived * pmax(asian_payoff, floor_locked)
```

```r
# 4.  DISCOUNTED PRICE & CONFIDENCE INTERVAL
```

```r
#    V_0 = exp(-r T) * E^Q[ P_T ]
#    Estimated by the sample mean over N_sim paths.
discount <- exp(-r * T_mat)
V0       <- discount * mean(payoff)
se       <- discount * sd(payoff) / sqrt(N_sim)

cat("\n----------------------------------------------------------------\n")
```

```
##
## ----------------------------------------------------------------
```

```r
cat("  PRICING RESULTS\n")
```

```
##   PRICING RESULTS
```

```r
cat("----------------------------------------------------------------\n")
```

```
## ----------------------------------------------------------------
```

```r
cat(sprintf("  Monte Carlo Price V0     = %.4f\n", V0))
```

```
##   Monte Carlo Price V0     = 15.1865
```

```r
cat(sprintf("  Standard Error           = %.4f\n", se))
```

```
##   Standard Error           = 0.0605
```

```r
cat(sprintf("  95%% Confidence Interval  = [%.4f, %.4f]\n",
            V0 - 1.96 * se, V0 + 1.96 * se))
```

```
##   95% Confidence Interval  = [15.0678, 15.3051]
```

```r
cat("----------------------------------------------------------------\n\n")
```

```
## ----------------------------------------------------------------
```

```r
# 5.  DIAGNOSTICS & DECOMPOSITION
```

```r
# How often does the lock-in floor dominate vs. the Asian payoff?
lock_in_dominates <- barrier_survived & (floor_locked > asian_payoff) &
  (floor_locked > 0)
asian_dominates   <- barrier_survived & (asian_payoff >= floor_locked) &
  (asian_payoff > 0)
both_zero         <- barrier_survived & (payoff == 0)

cat("  PAYOFF DECOMPOSITION (among surviving paths):\n")
```

```
##   PAYOFF DECOMPOSITION (among surviving paths):
```

```r
cat(sprintf("    Lock-in floor dominates : %s paths (%.1f%%)\n",
            format(sum(lock_in_dominates), big.mark = ","),
            100 * mean(lock_in_dominates)))
```

```
##     Lock-in floor dominates : 43,052 paths (43.1%)
```

```r
cat(sprintf("    Asian payoff dominates  : %s paths (%.1f%%)\n",
            format(sum(asian_dominates), big.mark = ","),
            100 * mean(asian_dominates)))
```

```
##     Asian payoff dominates  : 35,479 paths (35.5%)
```

```r
cat(sprintf("    Both zero (OTM)         : %s paths (%.1f%%)\n",
            format(sum(both_zero), big.mark = ","),
            100 * mean(both_zero)))
```

```
##     Both zero (OTM)         : 4,542 paths (4.5%)
```

```r
# 6.  BENCHMARK: PLAIN VANILLA EUROPEAN CALL (BLACK-SCHOLES)
```

```r
d1_bs <- (log(S0 / K) + (r + 0.5 * sigma^2) * T_mat) / (sigma * sqrt(T_mat))
d2_bs <- d1_bs - sigma * sqrt(T_mat)
bs_price <- S0 * pnorm(d1_bs) - K * exp(-r * T_mat) * pnorm(d2_bs)

cat(sprintf("\n  BENCHMARK: Black-Scholes European Call = %.4f\n", bs_price))
```

```
##
##   BENCHMARK: Black-Scholes European Call = 13.0957
```

```r
cat(sprintf("  Exotic / Vanilla ratio                 = %.2f%%\n",
            100 * V0 / bs_price))
```

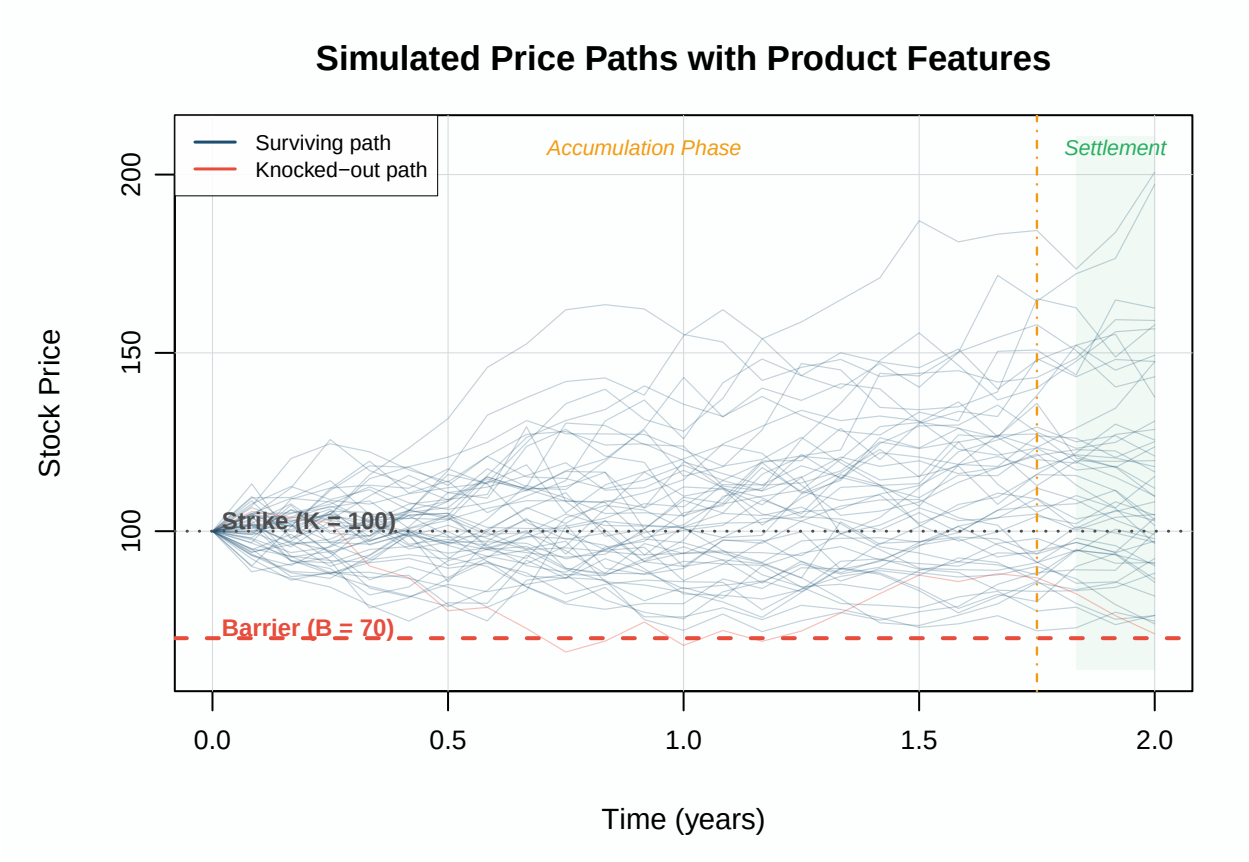
```
##   Exotic / Vanilla ratio                 = 115.97%
```

```r
cat("==============================================================\n")
```

```
## ==============================================================
```

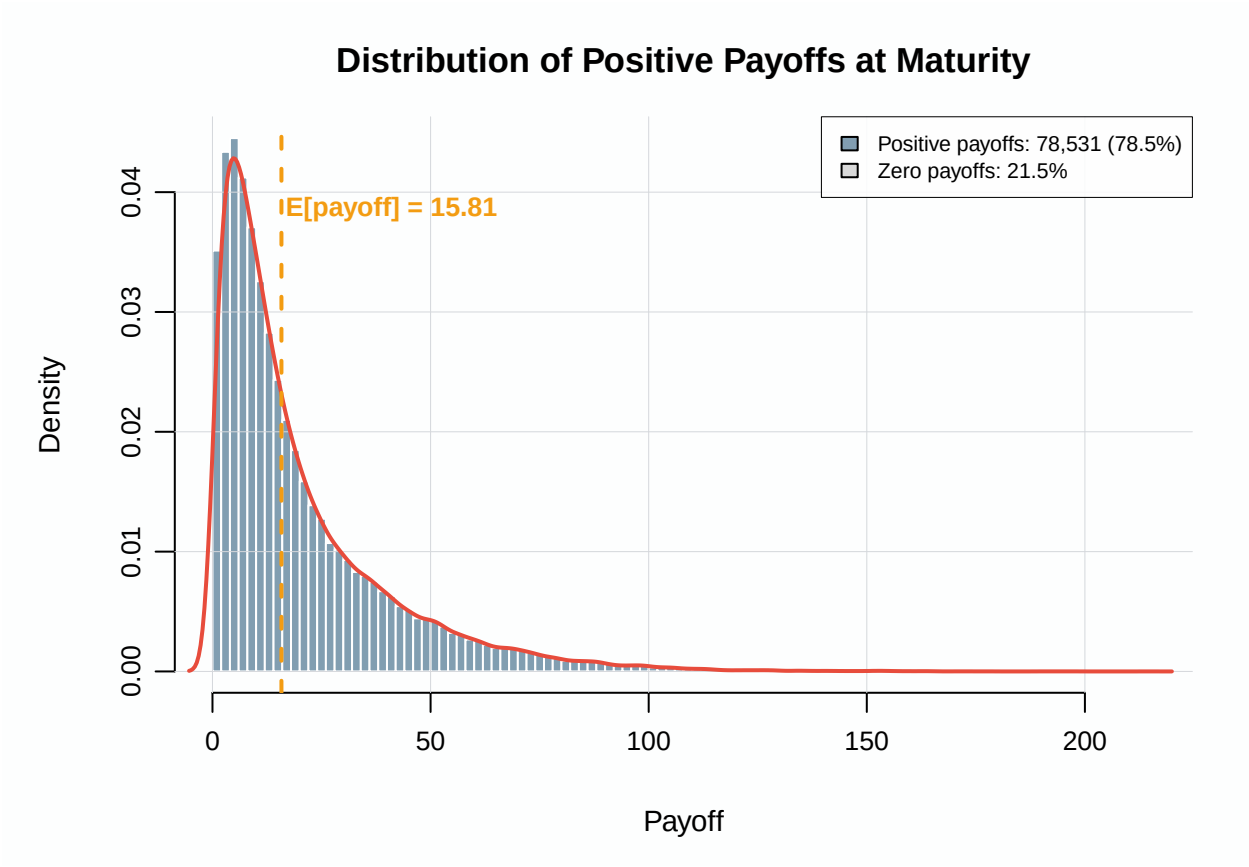
```r
# ######################################################################
# 7.  VISUALISATIONS
# ######################################################################

# -- Colour palette ----------------------------------------------------
col_main   <- "#1B4F72"
col_accent <- "#E74C3C"
col_asian  <- "#27AE60"
col_lock   <- "#F39C12"
col_barrier<- col_accent
col_grid   <- "#D5D8DC"
```

```r
col_bg     <- "#FDFEFE"

# Shared plot theme helper
setup_plot <- function(main, xlab, ylab, ...) {
  par(bg = col_bg, family = "sans", cex.main = 1.1, cex.lab = 0.95,
      cex.axis = 0.85, mar = c(4.5, 4.5, 3, 1.5), mgp = c(2.8, 0.7, 0))
}

# ======================================================================
# PLOT 1: Sample Price Paths with Barrier, Strike, and Phase Markers
# ======================================================================
setup_plot()
n_show <- 50
idx_show <- sample(N_sim, n_show)
S_plot <- cbind(S0, S[idx_show, ])      # prepend S0 as column 0
t_plot <- c(0, t_vec)

y_lo <- min(B - 5, min(S_plot) - 5)
y_hi <- max(S_plot) + 10

plot(NULL, xlim = c(0, T_mat), ylim = c(y_lo, y_hi),
     xlab = "Time (years)", ylab = "Stock Price",
     main = "Simulated Price Paths with Product Features")
grid(col = col_grid, lty = 1, lwd = 0.5)

# Background shading for settlement phase
rect(t_vec[22], y_lo, T_mat, y_hi, col = adjustcolor(col_asian, 0.06), border = NA)

# Draw paths, colour by knocked-out or surviving
for (i in seq_len(n_show)) {
  path_i <- S_plot[i, ]
  knocked <- any(path_i[-1] <= B)       # exclude S0
  col_i <- if (knocked) adjustcolor(col_accent, 0.35) else adjustcolor(col_main, 0.25)
  lines(t_plot, path_i, lwd = 0.6, col = col_i)
}

# Barrier and strike reference lines
abline(h = B, col = col_barrier, lwd = 2, lty = 2)
abline(h = K, col = "grey30", lwd = 1.5, lty = 3)
abline(v = t_vec[21], col = col_lock, lwd = 1.2, lty = 4)

text(T_mat * 0.01, B + 2.5, "Barrier (B = 70)", col = col_barrier, cex = 0.75, adj = 0, font = 2)
text(T_mat * 0.01, K + 2.5, "Strike (K = 100)", col = "grey30", cex = 0.75, adj = 0, font = 2)

# Phase labels
text(t_vec[11], y_hi - 3, "Accumulation Phase", col = col_lock, cex = 0.7, font = 3)
text(t_vec[23], y_hi - 3, "Settlement", col = col_asian, cex = 0.7, font = 3)

legend("topleft", legend = c("Surviving path", "Knocked-out path"),
       col = c(col_main, col_accent), lwd = 1.5, cex = 0.7,
       bg = adjustcolor("white", 0.85), box.lwd = 0.5)
```



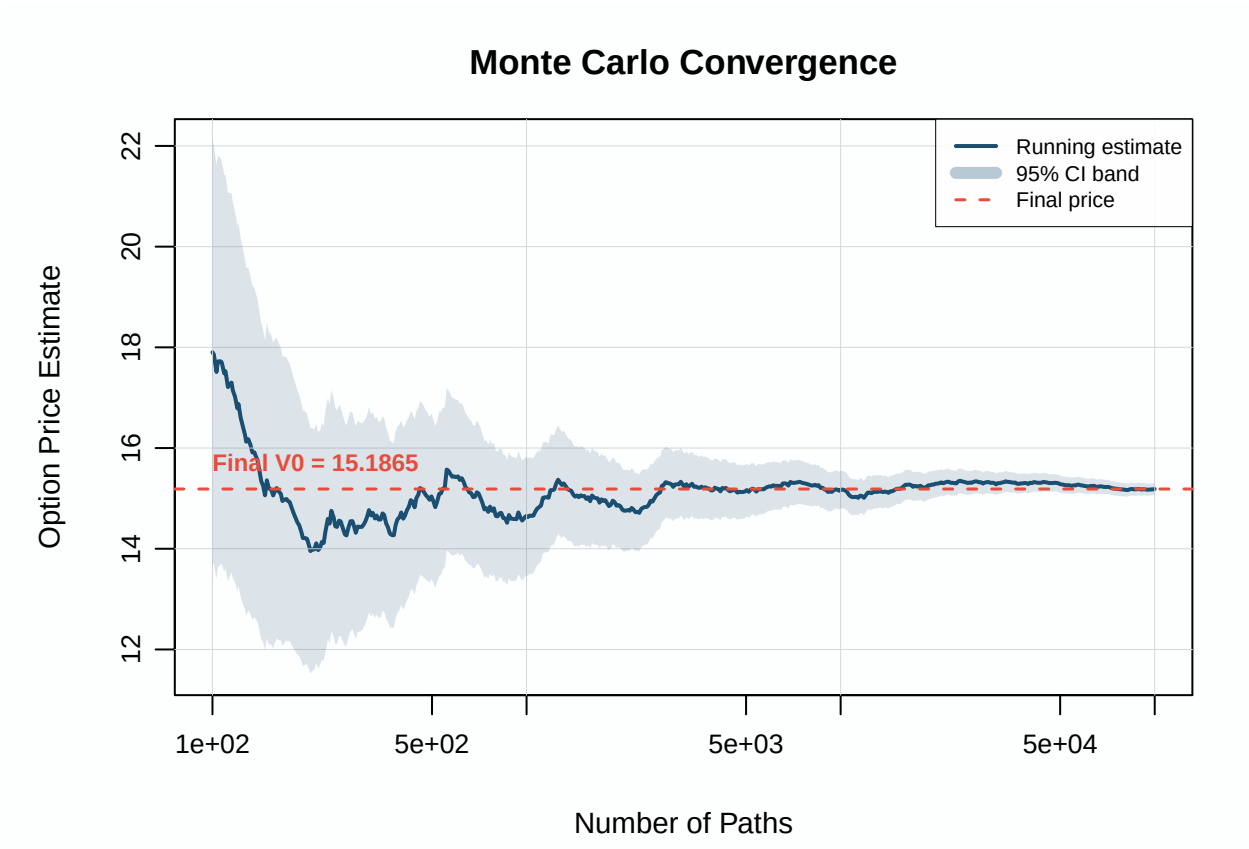

```r
# ====================================================================
# PLOT 2: Payoff Distribution (non-zero payoffs)
# ====================================================================
setup_plot()
pos_payoff <- payoff[payoff > 0]

hist(pos_payoff, breaks = 80, freq = FALSE,
     col = adjustcolor(col_main, 0.55), border = "white",
     main = "Distribution of Positive Payoffs at Maturity",
     xlab = "Payoff", ylab = "Density")
grid(col = col_grid, lty = 1, lwd = 0.5)

# Overlay a density curve
if (length(pos_payoff) > 10) {
  dens <- density(pos_payoff, adjust = 1.2)
  lines(dens, col = col_accent, lwd = 2)
}

abline(v = mean(payoff), col = col_lock, lwd = 2, lty = 2)
text(mean(payoff) + 1, max(dens$y) * 0.9,
     sprintf("E[payoff] = %.2f", mean(payoff)),
     col = col_lock, cex = 0.8, adj = 0, font = 2)

legend("topright",
       legend = c(sprintf("Positive payoffs: %s (%.1f%%)",
                          format(length(pos_payoff), big.mark = ","),
```

```
                        100 * length(pos_payoff) / N_sim),
                sprintf("Zero payoffs: %.1f%%",
                        100 * (1 - length(pos_payoff) / N_sim))),
       fill = c(adjustcolor(col_main, 0.55), "grey85"),
       cex = 0.7, bg = adjustcolor("white", 0.85), box.lwd = 0.5)
```



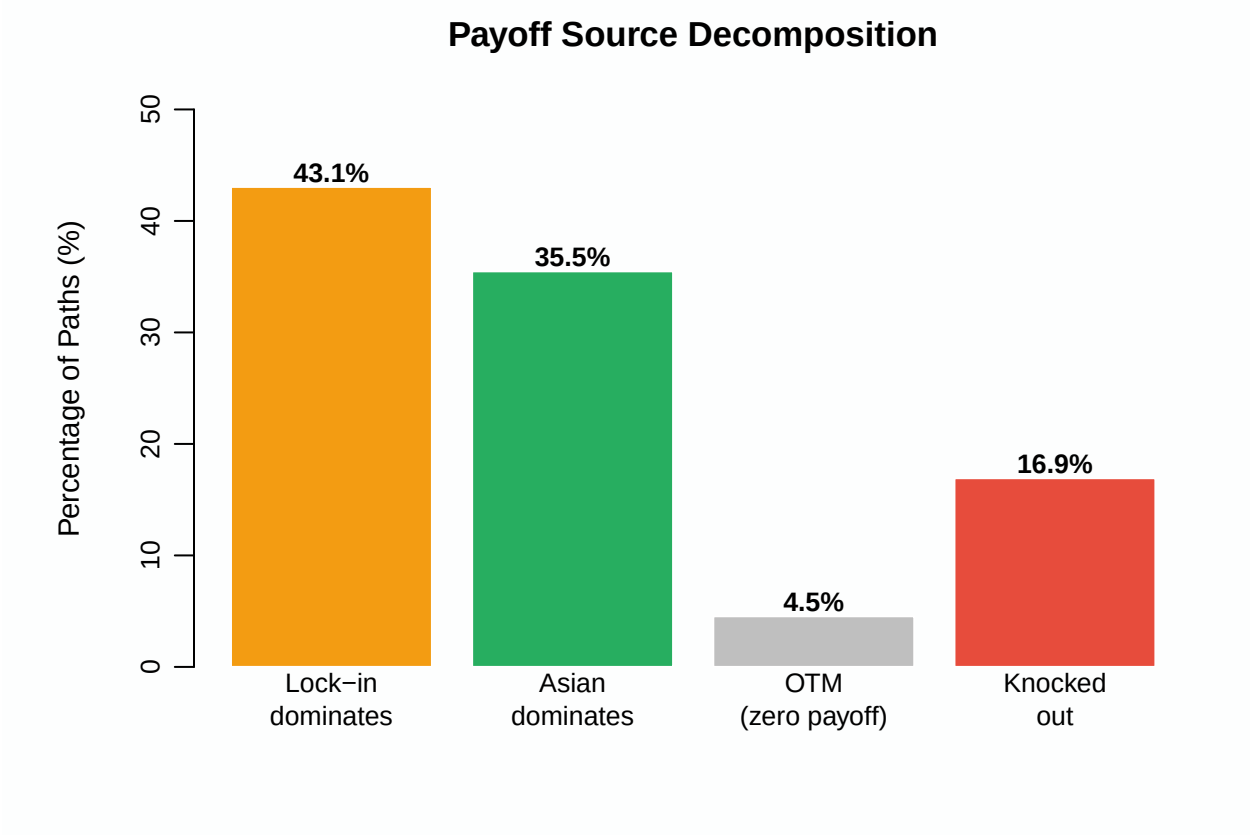

```
# ====================================================================
# PLOT 3: Monte Carlo Convergence of Option Price
# ====================================================================
setup_plot()
# Compute running mean of discounted payoff at logarithmically spaced steps
n_eval   <- 500
eval_pts <- unique(round(exp(seq(log(100), log(N_sim), length.out = n_eval))))
disc_pay <- discount * payoff

running_mean <- numeric(length(eval_pts))
running_se   <- numeric(length(eval_pts))
for (k in seq_along(eval_pts)) {
  n_k <- eval_pts[k]
  running_mean[k] <- mean(disc_pay[1:n_k])
  running_se[k]   <- sd(disc_pay[1:n_k]) / sqrt(n_k)
}

y_range <- range(c(running_mean - 1.96 * running_se,
                   running_mean + 1.96 * running_se))
```

```r
plot(eval_pts, running_mean, type = "l", lwd = 2, col = col_main,
     xlab = "Number of Paths", ylab = "Option Price Estimate",
     main = "Monte Carlo Convergence", ylim = y_range, log = "x")
grid(col = col_grid, lty = 1, lwd = 0.5)

# 95% confidence band
polygon(c(eval_pts, rev(eval_pts)),
        c(running_mean - 1.96 * running_se,
          rev(running_mean + 1.96 * running_se)),
        col = adjustcolor(col_main, 0.15), border = NA)

abline(h = V0, col = col_accent, lwd = 1.5, lty = 2)
text(eval_pts[1], V0 + diff(y_range) * 0.05,
     sprintf("Final V0 = %.4f", V0), col = col_accent, cex = 0.75, adj = 0, font = 2)

legend("topright", legend = c("Running estimate", "95% CI band", "Final price"),
       col = c(col_main, adjustcolor(col_main, 0.3), col_accent),
       lwd = c(2, 6, 1.5), lty = c(1, 1, 2), cex = 0.7,
       bg = adjustcolor("white", 0.85), box.lwd = 0.5)
```




```r
# ====================================================================
# PLOT 4: Payoff Source Breakdown -- Stacked Bar
# ====================================================================
setup_plot()
par(mar = c(4.5, 5, 3, 1.5))
```

```r
counts <- c(
  "Lock-in\ndominates"  = sum(lock_in_dominates),
  "Asian\ndominates"    = sum(asian_dominates),
  "OTM\n(zero payoff)"  = sum(both_zero),
  "Knocked\nout"         = n_knocked
)
pcts <- 100 * counts / N_sim

bp <- barplot(pcts, col = c(col_lock, col_asian, "grey75", col_accent),
              border = "white", ylim = c(0, max(pcts) * 1.2),
              ylab = "Percentage of Paths (%)",
              main = "Payoff Source Decomposition")
text(bp, pcts + max(pcts) * 0.03, sprintf("%.1f%%", pcts), cex = 0.85, font = 2)
```

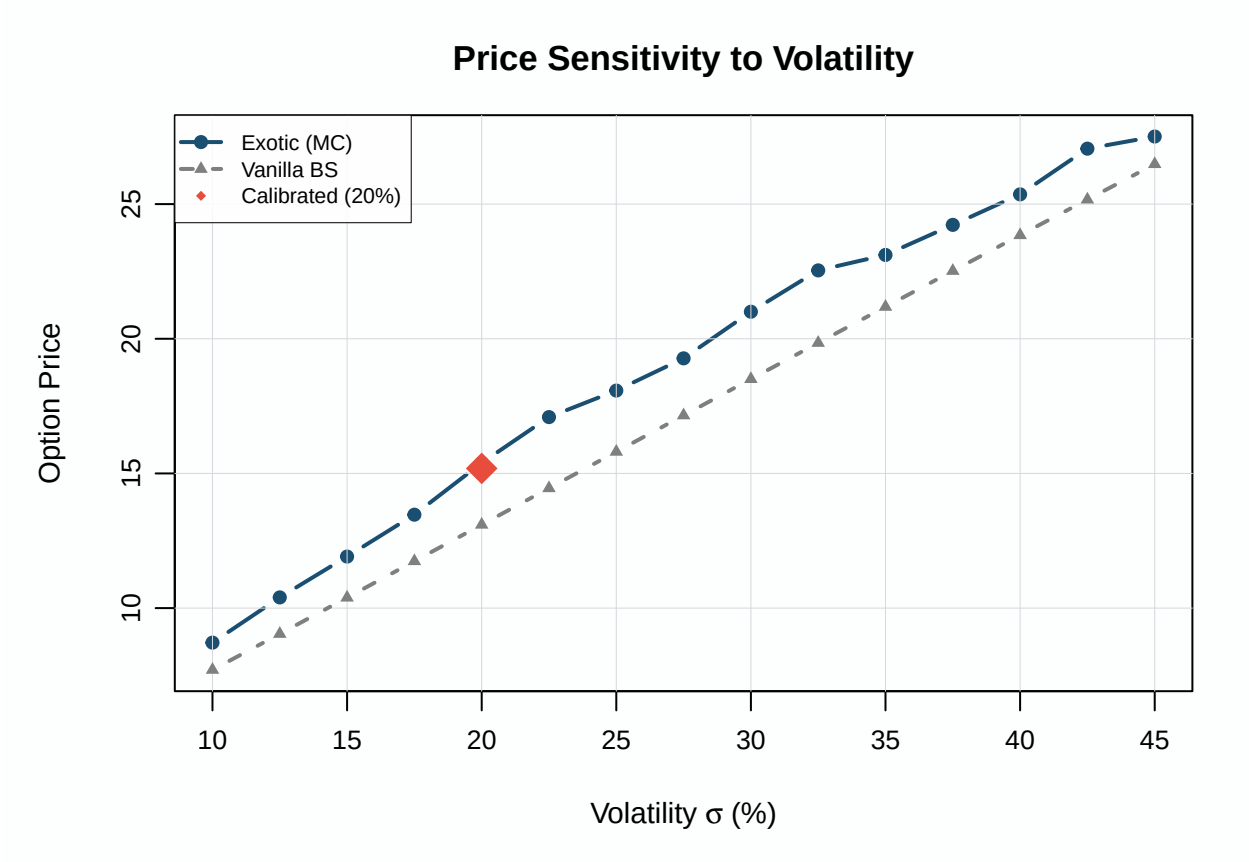



```r
# ======================================================================
# PLOT 5: Sensitivity -- Price vs Volatility (quick re-simulation)
# ======================================================================
setup_plot()
sigma_grid <- seq(0.10, 0.45, by = 0.025)
n_sens     <- 20000   # fewer paths for speed
price_sens <- numeric(length(sigma_grid))

cat("\n  Running volatility sensitivity (", n_sens, " paths each)...\n")
```

```
##
```

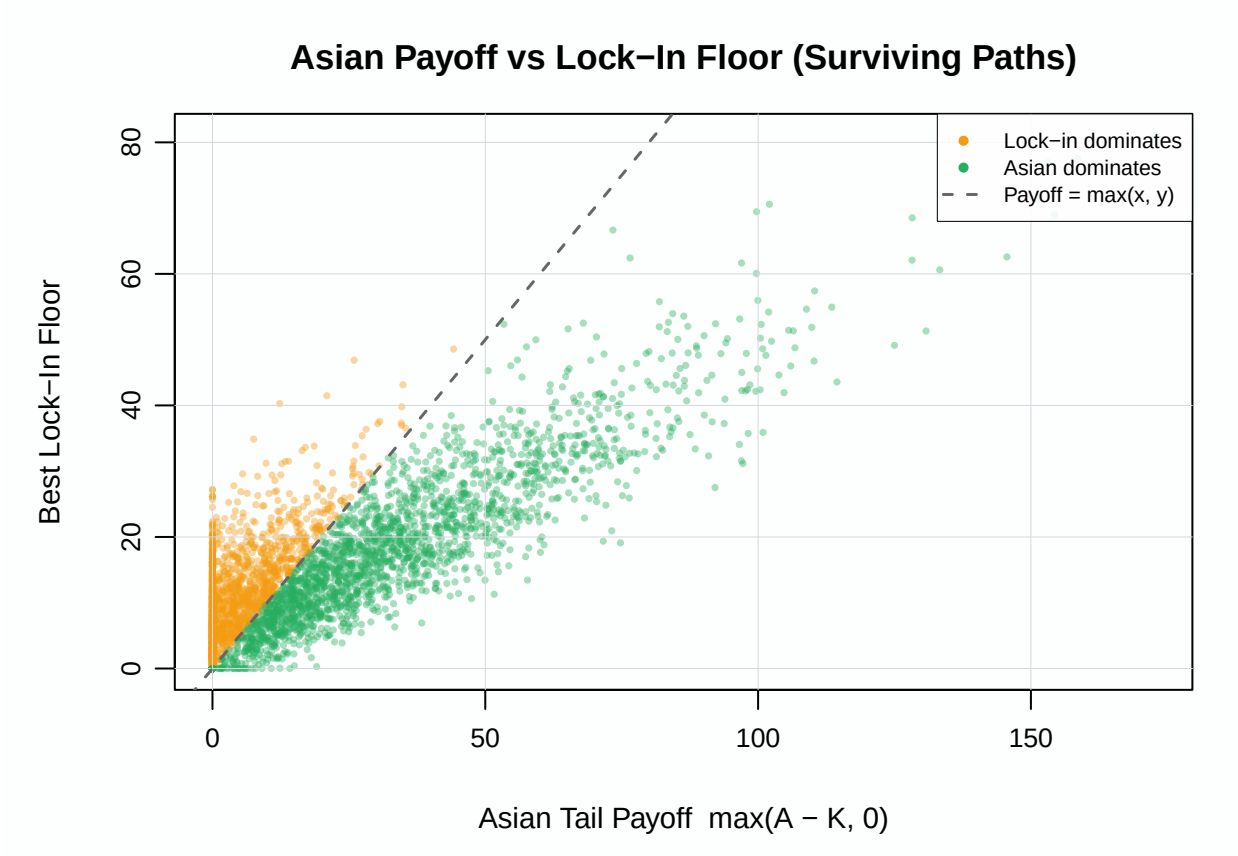
```
##   Running volatility sensitivity ( 20000  paths each)...
```

```r
for (s in seq_along(sigma_grid)) {
  sig_s <- sigma_grid[s]
  Z_s   <- matrix(rnorm(n_sens * M), nrow = n_sens, ncol = M)
  lr_s  <- (r - 0.5 * sig_s^2) * dt + sig_s * sqrt(dt) * Z_s
  lS_s  <- t(apply(lr_s, 1, cumsum))
  S_s   <- S0 * exp(lS_s)

  pmin_s  <- apply(S_s, 1, min)
  surv_s  <- (pmin_s > B)
  lock_s  <- apply(beta * pmax(S_s[, 1:21] - K, 0), 1, max)
  A_s     <- rowMeans(S_s[, 22:24])
  asian_s <- pmax(A_s - K, 0)
  pay_s   <- surv_s * pmax(asian_s, lock_s)

  price_sens[s] <- discount * mean(pay_s)
}

# Also compute BS vanilla for the same grid
bs_sens <- numeric(length(sigma_grid))
for (s in seq_along(sigma_grid)) {
  sig_s <- sigma_grid[s]
  d1_s  <- (log(S0 / K) + (r + 0.5 * sig_s^2) * T_mat) / (sig_s * sqrt(T_mat))
  d2_s  <- d1_s - sig_s * sqrt(T_mat)
  bs_sens[s] <- S0 * pnorm(d1_s) - K * exp(-r * T_mat) * pnorm(d2_s)
}

y_rng <- range(c(price_sens, bs_sens))

plot(sigma_grid * 100, price_sens, type = "b", pch = 19, cex = 0.7,
     lwd = 2, col = col_main,
     xlab = expression("Volatility " * sigma * " (%)"),
     ylab = "Option Price",
     main = "Price Sensitivity to Volatility",
     ylim = y_rng)
grid(col = col_grid, lty = 1, lwd = 0.5)

lines(sigma_grid * 100, bs_sens, type = "b", pch = 17, cex = 0.7,
      lwd = 2, col = "grey50", lty = 2)

# Mark the calibrated point
points(sigma * 100, V0, pch = 18, cex = 2.2, col = col_accent)

legend("topleft",
       legend = c("Exotic (MC)", "Vanilla BS", sprintf("Calibrated (%.0f%%)", sigma * 100)),
       col = c(col_main, "grey50", col_accent),
       pch = c(19, 17, 18), lwd = c(2, 2, NA), lty = c(1, 2, NA),
       cex = 0.7, bg = adjustcolor("white", 0.85), box.lwd = 0.5)
```




```r
# ====================================================================
# PLOT 6: Terminal Asian Average vs Lock-In Floor (scatter, surviving)
# ====================================================================
setup_plot()
surv_idx <- which(barrier_survived)
n_scatter <- min(5000, length(surv_idx))
samp_idx <- sample(surv_idx, n_scatter)

# Determine which component dominated for colouring
dom_col <- ifelse(floor_locked[samp_idx] > asian_payoff[samp_idx],
                  adjustcolor(col_lock, 0.4),
                  adjustcolor(col_asian, 0.4))

plot(asian_payoff[samp_idx], floor_locked[samp_idx],
     pch = 16, cex = 0.5, col = dom_col,
     xlab = "Asian Tail Payoff  max(A - K, 0)",
     ylab = "Best Lock-In Floor",
     main = "Asian Payoff vs Lock-In Floor (Surviving Paths)")
grid(col = col_grid, lty = 1, lwd = 0.5)

# 45-degree line
abline(a = 0, b = 1, col = "grey40", lwd = 1.5, lty = 2)

legend("topright",
       legend = c("Lock-in dominates", "Asian dominates", "Payoff = max(x, y)"),
       col = c(col_lock, col_asian, "grey40"),
```

```
       pch = c(16, 16, NA), lty = c(NA, NA, 2), lwd = c(NA, NA, 1.5),
       cex = 0.7, bg = adjustcolor("white", 0.85), box.lwd = 0.5)
```




```
cat("\n  [OK] All 6 plots generated.\n")
```

```
##
##   [OK] All 6 plots generated.
```

```
cat("==============================================================\n")
```

```
## ==============================================================
```